

metasignal: A Python Package for Comprehensive Metacognitive Analysis and Decision-Making

Saurabh Ranjan¹, Mukesh Makwana², Konstantina Sokratous³, and Brian Odegaard¹

¹ University of Florida, USA ² Brown University, USA ³ University of Missouri, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

metasignal is an open-source Python package for signal detection theory (SDT) and metacognitive measurement. It implements the 17 metacognitive measures evaluated by Rahnev (2025), together with the reference variables d' (perceptual sensitivity), response criterion c (response bias), and mean confidence. The 17 measures comprise three meta- d' -family estimates, *meta- d'* , *M-ratio*, and *M-difference*; four nonparametric Type-2 measures, the Type-2 area under the receiver-operating-characteristic curve (*AUC2*), *Gamma*, *Phi*, and *delta confidence*, together with their eight SDT-normalized ratio and difference forms; and two model-based measures, *meta-noise* and *meta-uncertainty*. A single function computes the complete set from trial-level stimulus, response, and confidence arrays. The package also provides a command-line interface, group summaries, bootstrap confidence intervals, permutation tests, optional hierarchical Bayesian models, and information-theoretic measures. metasignal unifies these measures in a single platform to encourage broader metacognition research and adoption in decision-making studies.

Statement of Need

Metacognition is the ability to evaluate the quality of one's own decisions. It supports learning, adaptive choice, and communication of uncertainty (Fleming & Dolan, 2012; Yeung & Summerfield, 2012). Metacognitive measurement is also relevant to clinical research (Hoven et al., 2019), behavioral economics (Moore & Healy, 2008), and the evaluation of uncertainty reported by artificial-intelligence systems, where metacognitive sensitivity has been proposed as key to calibrating human trust in AI (Griot et al., 2025; D. Lee et al., 2025; Steyvers & Peters, 2025). A consensus statement identified reliable computational models and standardized measurement as major goals for visual-metacognition research (Rahnev et al., 2022).

Numerous metacognitive measures have been proposed, but they quantify distinct properties and vary in their sensitivity to task performance, response bias, confidence bias, and sample size. Among these, meta- d' is widely used because it expresses metacognitive sensitivity in signal detection theory (SDT) units directly comparable to perceptual sensitivity (Maniscalco & Lau, 2012). In plain terms, it measures how well confidence tracks actual accuracy, on the same scale researchers already use for perceptual sensitivity. Although the original MATLAB implementation (Maniscalco & Lau, 2014) and its Python port (A. L. F. Lee, 2018) brought meta- d' to a broad audience, both tools focus exclusively on this single measure. The other sixteen measures benchmarked by Rahnev (2025) remained scattered across isolated codebases, leaving no single Python package that spans the full set. This fragmentation has hindered researchers from comparing metacognitive measures within a single, reproducible workflow.

Rahnev (2025) provided the first broad empirical comparison of 17 measures. The study assessed validity and precision, dependence on nuisance variables, split-half reliability, and

test-retest reliability. `metasignal` translates this benchmark into a documented Python package that requires no proprietary software. It is intended for cognitive scientists, psychophysicists, neuroscientists, clinical researchers, and researchers studying confidence in artificial systems.

State of the Field

The MATLAB toolbox by Maniscalco & Lau (2014) and its Python port (A. L. F. Lee, 2018) remain important reference implementations for standard meta- d' estimation. Packages like `metadpy` (Legrand, 2021) and `HMeta-d` (Fleming, 2017) introduced valuable Bayesian and hierarchical estimation frameworks, though they remain focused primarily on the meta- d' family. In R, `statConfR` (Rausch et al., 2025) provides confidence models alongside information-theoretic metrics. While each of these tools excels within its intended scope, none offers the full battery of metrics benchmarked by Rahnev (2025) through a single, native Python interface. `metasignal` addresses this gap by bringing an expansive collection of metacognitive measures under a single Python package umbrella. Rather than replacing existing tools, it complements them by providing broad metric coverage and a standardized trial-level API, which together lower the barrier to wider adoption and more rigorous metacognition research. The optional `sdtbayes` subpackage enables Stan-based hierarchical estimation, and the experimental `itmc` subpackage implements measures grounded in metacognitive information theory (Dayan, 2023; Meyen et al., 2025).

Software Design

`metasignal` is built around four primary design principles. First, input data (stimulus identity, behavioral response, and confidence) are structured as tabular trial-level data compatible with standard Python processing workflows. Second, the core `stdpy` layer relies solely on NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020), providing a pure-Python implementation. Third, it offers flexible execution: `compute_all_measures` provides a single entry point for broad profiling, while individual functions remain exposed for focused queries. Fourth, complex Bayesian dependencies are isolated from the lightweight frequentist core.

When invoked, `compute_all_measures` returns 26 values. The first 20 span d' , decision criterion, mean confidence, and the 17 metacognitive metrics. The final six outputs (log-likelihood, AIC, BIC, AICc, fitted parameter count, and sample size) serve as diagnostic parameters for the meta- d' fit rather than distinct metacognitive measures. An optional `return_type` argument ('dict' or 'dataframe') labels these same 26 values by measure name instead of position; the default 'array' preserves positional, backward-compatible output.

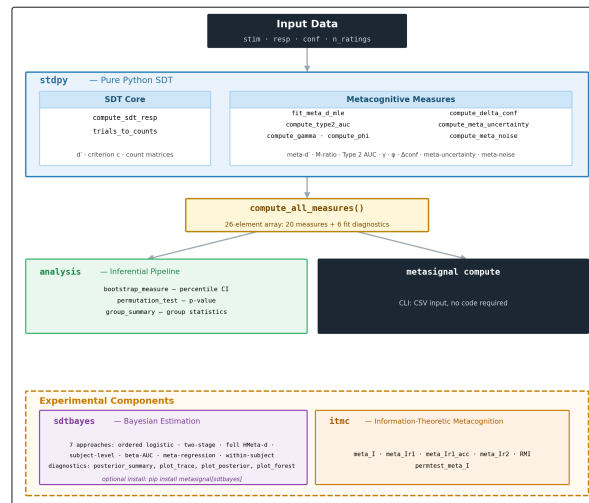


Figure 1: Architecture of metasignal. Trial-level stimulus, response, confidence, and rating-scale arrays (*stim*, *resp*, *conf*, *n_ratings*) enter the stable *stdpy* core, which computes Type-1 SDT quantities (*d'*, criterion *c*) and all 17 metacognitive measures; *compute_all_measures* packages these into a single 26-element array (20 measures plus 6 meta-*d'* fit diagnostics). This output feeds the analysis layer (bootstrap confidence intervals, permutation tests, group summaries) and the *metasignal* compute command-line interface, which accepts CSV input without requiring code. The dashed box shows optional, dependency-isolated extensions: *sdtbayes* for hierarchical Bayesian estimation (seven modeling approaches) and *itmc* for information-theoretic metacognition measures.

Methods

Core computation

The package first converts trial-level data into Type-2 response-count arrays. Here, “Type-1” denotes the original stimulus/response decision and “Type-2” the confidence judgment about it, unrelated to Type I/Type II statistical error. Type-1 sensitivity and criterion are computed under the equal-variance SDT model (Green & Swets, 1966). Nonparametric Type-2 measures are computed directly from the observed counts. Meta-*d'* is estimated by maximum likelihood (Maniscalco & Lau, 2012), from which M-ratio and M-difference are derived. SDT-normalized Ratio and Difference measures compare observed statistics with statistics expected under an ideal equal-variance SDT observer.

Meta-noise is estimated with the lognormal confidence-noise model used by the MATLAB benchmark. The Python implementation follows the MATLAB procedure: it evaluates the zero-noise Gaussian baseline, searches outward from that lower bound, uses golden-section optimization, and evaluates the precomputed integral table using inverse-distance weighting. Meta-uncertainty is fitted with its corresponding model-based estimator.

Validation data and procedure

Validation used the six datasets distributed with the Rahnev analysis pipeline: Haddara, Maniscalco, Rouault experiments 1 and 2, Shekhar, and Locke. The comparison used three sources:

1. values reported in the text, figures, and supplementary tables of Rahnev (2025);
2. subject-level MATLAB arrays stored in `matlab/metasignal_mat/Results`; and
3. Python arrays generated from the same trial subsets.

Ten subject-level analysis arrays were compared: raw estimates, metacognitive-bias recoding,

98 odd-even splits, difficulty conditions, and response-bias conditions. For each measure we
99 examined Pearson correlation, mean absolute error, root-mean-square error, signed bias,
100 maximum absolute error, and missing-value agreement. We also reproduced 19 statistical
101 checks from the supplementary analyses and compared the 17-measure profiles for task-
102 performance dependence, metacognitive-bias dependence, response-bias dependence, and
103 test-retest reliability. The test-retest comparison used the same bin size (400 trials), day
104 structure, and Fisher-z aggregation as the MATLAB scripts. Precision was also recomputed
105 directly from raw trial data using the bin-stratified protocol Rahnev (2025) describes (non-
106 overlapping 50-, 100-, 200-, and 400-trial bins), under a bin-instance cap required for
107 computational tractability (see Limitations).

108 The complete workflow is implemented in `analysis/rahnev_comparison/scripts`. The JSON
109 results, CSV summaries, identity plots, profile overlays, and combined PDFs are stored beside
110 the scripts. This makes the validation repeatable rather than dependent on a manually prepared
111 table.

DRAFT

Validation Results

metasignal validation: Rahnev (2025) · MATLAB pipeline · Python package

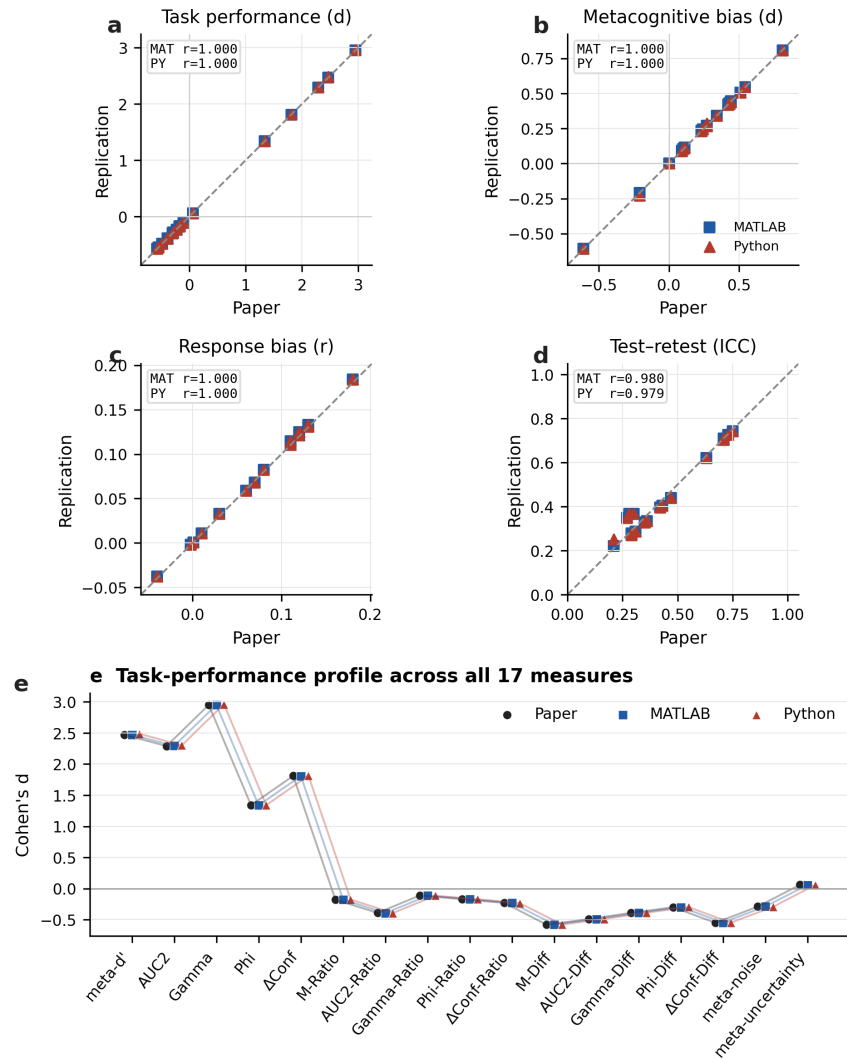


Figure 2: Cross-implementation validation against Rahnev (2025). In each panel, published values (x-axis, "Paper") are plotted against replication values (y-axis) from the MATLAB pipeline (blue squares) and the Python metasignal package (red triangles); the dashed line marks exact identity. MAT and PY report the Pearson correlation between each replication and the published values. **a**, Task-performance effect sizes (Cohen's d ; MAT $r = 1.000$, PY $r = 1.000$). **b**, Metacognitive-bias effect sizes (Cohen's d ; MAT $r = 1.000$, PY $r = 1.000$). **c**, Response-bias correlations (r ; MAT $r = 1.000$, PY $r = 1.000$). **d**, Test-retest reliability (intraclass correlation coefficient, ICC; MAT $r = 0.980$, PY $r = 0.979$). **e**, Task-performance profile (Cohen's d) across all 17 metacognitive measures, comparing published values (black circles) with MATLAB (blue squares) and Python (red triangles) replications.

All 10 subject-level comparison arrays passed the predefined comparison gate, and all 19 supplementary statistical checks matched in statistical significance and reference t value. Across the 18 non-model-based measures, MATLAB and Python agreed to numerical precision at the subject level (maximum systematic bias below 1.5×10^{-3} ; per-analysis correlations of

117 1.000). Ten of these measures agreed to within a maximum absolute difference of 10^{-2} across
118 every subject and analysis.

119 The analysis profiles were essentially identical across sources: task-performance dependence
120 (paper-Python $r = 1.000$; MATLAB-Python $r = 1.000$), metacognitive-bias dependence
121 (paper-Python $r = 1.000$; MATLAB-Python $r = 1.000$), and response-bias dependence
122 (paper-Python $r = 1.000$; MATLAB-Python $r = 1.000$). Test-retest ICC (intraclass
123 correlation coefficient), matched to the MATLAB bin-size-400 protocol, also aligned closely
124 (MATLAB-Python $r = 0.999$).

125 Some differences remain and are reported as limitations rather than hidden. Meta-d', M-ratio,
126 and M-difference correlate at 1.000 across analyses but can differ by up to about 0.14 for
127 individual participants because both pipelines use bounded numerical maximum-likelihood
128 optimizers with different internal solvers. Meta-noise and meta-uncertainty retain a small
129 number of subject-level differences in the sparsest difficulty subsets, where the likelihood surface
130 is flat. A single degenerate value returned by the MATLAB meta-noise search (near 0.4959) is
131 identified programmatically and excluded from agreement statistics. For users who need to
132 reproduce a specific MATLAB result file rather than a scientifically stable estimate, the model-
133 based estimators expose an optional `matlab_compat` mode that mimics MATLAB's single-start
134 optimizer configuration. The package therefore supports the scientific conclusions of Rahnev
135 (2025) and reproduces the MATLAB pipeline to numerical precision for the non-model-based
136 measures, while documenting the residual, well-understood variation in low-information model
137 fits.

138 Simple Procedure for Use

139 Installation

140 Install from the public repository:

```
pip install git+https://github.com/saurabhr/metasignal.git
```

141 For development or reproducible validation:

```
git clone https://github.com/saurabhr/metasignal.git  
cd metasignal  
pip install -e .
```

142 Prepare trial-level data

143 Each trial requires four values:

- 144 ▪ `stim`: stimulus category, coded with two values such as 0 and 1;
- 145 ▪ `resp`: participant response, coded with the same two values;
- 146 ▪ `conf`: integer confidence rating from 1 to `n_ratings`; and
- 147 ▪ `n_ratings`: number of possible confidence levels.

148 The arrays must have the same length. Missing trials are removed jointly. A study should
149 compute participant-level estimates separately before group inference.

150 Compute all measures

151 The following self-contained example simulates one participant with a fixed random seed, so
152 that any user can run it and obtain the same numbers. This doubles as a minimal reproducibility
153 check of the installation.

```
import numpy as np  
from metasignal import stdpy
```

```
rng = np.random.default_rng(2025)
n = 800
stim = rng.integers(0, 2, n) # two stimulus categories
evidence = (stim * 2 - 1) * 0.9 + rng.normal(0, 1, n)
resp = (evidence > 0).astype(int) # observer response
edges = np.quantile(np.abs(evidence), [0.25, 0.5, 0.75])
conf = np.digitize(np.abs(evidence), edges) + 1 # 4-point confidence

values = stdpy.compute_all_measures(stim, resp, conf, n_ratings=4)

meta_d = values[0] # meta-d'
auc2 = values[1] # Type-2 AUC
m_ratio = values[5] # M-ratio (meta-d'/d')
dprime = values[17] # d'
print(f"d'={dprime:.3f} meta-d'={meta_d:.3f} M-ratio={m_ratio:.3f} AUC2={auc2:.3f}")
```

154 Running this prints:

```
d'=1.929 meta-d'=1.841 M-ratio=0.955 AUC2=0.762
```

155 compute_all_measures returns all 26 values in a fixed order; the first 20 are the 17
156 metacognitive measures plus d', criterion, and mean confidence, and the last six are meta-d'
157 fit diagnostics. Researchers should use realistic trial counts for scientific estimation; the seed
158 here only makes the example reproducible.

159 **Compute selected measures**

```
dprime, criterion, ln_beta = stdpy.compute_sdt_resp(stim, resp)
nr_s1, nr_s2 = stdpy.trials_to_counts(stim, resp, conf, n_ratings=4)
meta_fit = stdpy.fit_meta_d_mle(nr_s1, nr_s2)
auc2 = stdpy.compute_type2_auc(nr_s1, nr_s2)

print(meta_fit["meta_da"])
print(meta_fit["M_ratio"])
```

160 **Command-line use**

```
metasignal compute \
  --stim "0,1,0,1,0,1,0,1" \
  --resp "0,1,0,0,0,1,1,1" \
  --conf "4,4,3,1,3,4,2,3" \
  --n-ratings 4
```

161 Trial data can also be read from a CSV with one trial per row instead of typed inline:

```
cat > trials.csv <<'CSV'
stim,resp,conf
0,0,4
1,1,4
0,0,3
1,0,1
0,0,3
1,1,4
0,1,2
1,1,3
CSV
```

```
metasignal compute --csv trials.csv --n-ratings 4
```

Both commands above print the identical 26-value table (they encode the same eight trials); this was verified directly rather than assumed. Column names default to `stim`, `resp`, and `conf` and can be overridden with `--stim-col`, `--resp-col`, and `--conf-col` for CSVs with different headers.

Recommended scientific workflow

1. Check that stimulus and response each contain two categories.
2. Confirm that confidence values are integers within the declared scale.
3. Compute measures independently for each participant and condition.
4. Inspect missing values and convergence diagnostics before group analysis.
5. Use bootstrap intervals or permutation tests when sampling distributions are uncertain.
6. Report the measure definition, trial count, confidence scale, exclusion criteria, and reliability protocol.
7. For direct replication, preserve the original bin sizes, recoding rules, and random-resampling procedure.

Research Impact Statement

`metasignal` turns a broad methodological benchmark into reusable research infrastructure. It provides a common implementation for comparing measures, lowers the barrier for laboratories without MATLAB, and makes numerical validation visible and reproducible. Seven tutorial notebooks cover preprocessing, measure computation, statistical tables, difficulty dependence, bias, split-half reliability, and test-retest reliability. Automated tests, continuous integration, API documentation, and contribution guidelines support reuse and extension.

The validation materials are also a scholarly contribution. They identify implementation details that materially affect results, especially the zero-noise boundary in meta-noise fitting and the use of proportions in SDT-expected ratings. Recording these details helps prevent apparently contradictory results that arise from software rather than theory.

Limitations

Validation Scope

No single metacognitive measure is optimal for every experimental design (Rahnev, 2025). Model-based and ratio measures can become unstable with small sample sizes or sparse confidence categories, particularly when expected denominators approach zero. Users should select measures based on their scientific questions and report analysis settings completely.

While `metasignal` perfectly reproduces the core task-performance, metacognitive-bias, and response-bias benchmark profiles from Rahnev (2025) ($r = 1.000$), precision validation requires nuanced interpretation. Precision was recomputed directly from raw trial data using the bin-stratified protocol Rahnev (2025) describes (non-overlapping 50-, 100-, 200-, and 400-trial bins). Under a bin-instance cap required for computational tractability, precision estimates display elevated sampling noise, occasionally causing unstable measures (e.g., Gamma-Ratio) to invert signs. Consequently, current precision figures should be treated as illustrative rather than an exact protocol match to Rahnev (2025).

Conclusions

`metasignal` provides a simple, open, and validated Python interface to the principal SDT and metacognitive measures compared by Rahnev (2025). That numerical fidelity means researchers can adopt `metasignal` in place of the MATLAB pipeline without sacrificing comparability to

the published benchmark. The package combines breadth, ease of use, transparent validation, and optional inferential tools, and remaining differences are localized to low-information model fits or unmatched reliability protocols and are documented explicitly. Taken together, these properties make metasignal suitable for reproducible metacognition research while preserving appropriate scientific caution.

Code Availability

metasignal is open source (MIT license) and available at <https://github.com/saurabhr/metasignal>. Full documentation, including the API reference and tutorials, is hosted at <https://metasignal.readthedocs.io/en/latest/>. The complete Rahnev (2025) replication and validation workflow (comparison scripts, JSON/CSV results, and figures) is included in the repository under analysis/rahnev_comparison/, and the bundled MATLAB reference pipeline and results are under matlab/metasignal_mat/. The original MATLAB code for the 17 metacognition measures, from which the bundled MATLAB pipeline is derived, is available at <https://osf.io/y5w2d/> (Rahnev, 2025).

AI Usage Disclosure

Claude (Anthropic) and OpenAI language models were used to assist with initial drafts of portions of the source code, documentation, validation report, and manuscript. All software behavior, numerical comparisons, citations, scientific claims, and final wording were reviewed and edited by the authors. Responsibility for the software and manuscript remains with the authors.

References

- Dayan, P. (2023). Metacognitive information theory. *Open Mind*, 7, 392–411. https://doi.org/10.1162/opmi_a_00091
- Fleming, S. M. (2017). HMeta-d: Hierarchical bayesian estimation of metacognitive efficiency from confidence ratings. *Neuroscience of Consciousness*, 2017(1), nix007. <https://doi.org/10.1093/nc/nix007>
- Fleming, S. M., & Dolan, R. J. (2012). The neural basis of metacognitive ability. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 367(1594), 1338–1349. <https://doi.org/10.1098/rstb.2011.0417>
- Green, D. M., & Swets, J. A. (1966). *Signal detection theory and psychophysics*. Wiley.
- Griot, M., Hemptinne, C., Vanderdonckt, J., & Yuksel, D. (2025). Large language models lack essential metacognition for reliable medical reasoning. *Nature Communications*, 16(1), 642. <https://doi.org/10.1038/s41467-024-55628-6>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hoven, M., Lebreton, M., Engelmann, J. B., Denys, D., Luigjes, J., & Holst, R. J. van. (2019). Abnormalities of confidence in psychiatry: An overview and future perspectives. *Translational Psychiatry*, 9(1), 268. <https://doi.org/10.1038/s41398-019-0602-7>
- Lee, A. L. F. (2018). *Meta-d' analysis in Python*. <https://sites.google.com/view/alanflee-lab/resources>

- 248 Lee, D., Pruitt, J., Zhou, T., Du, J., & Odegaard, B. (2025). Metacognitive sensitivity: The
249 key to calibrating trust and optimal decision making with AI. *PNAS Nexus*, 4(5), pgaf133.
250 <https://doi.org/10.1093/pnasnexus/pgaf133>
- 251 Legrand, N. (2021). *Metadpy: Bayesian meta-d' for python*. [https://github.com/LegrandNico/](https://github.com/LegrandNico/metadpy)
252 [metadpy](https://github.com/LegrandNico/metadpy).
- 253 Maniscalco, B., & Lau, H. (2012). A signal detection theoretic approach for estimating
254 metacognitive sensitivity from confidence ratings. *Consciousness and Cognition*, 21(1),
255 422–430. <https://doi.org/10.1016/j.concog.2011.09.021>
- 256 Maniscalco, B., & Lau, H. (2014). A signal detection theoretic approach for estimating
257 metacognitive sensitivity from confidence ratings. [http://www.columbia.edu/~bsm2105/](http://www.columbia.edu/~bsm2105/type2sdt/)
258 [type2sdt/](http://www.columbia.edu/~bsm2105/type2sdt/).
- 259 Meyen, S., Göppert, F., Schrenk, C., Luxburg, U. von, & Franz, V. H. (2025). Information-
260 theoretic measures of metacognition: Bounds and relation to group performance. *Open*
261 *Mind*, 9, 1728–1762. <https://doi.org/10.1162/OPMI.a.40>
- 262 Moore, D. A., & Healy, P. J. (2008). The trouble with overconfidence. *Psychological Review*,
263 115(2), 502–517. <https://doi.org/10.1037/0033-295X.115.2.502>
- 264 Rahnev, D. (2025). A comprehensive assessment of current methods for measuring
265 metacognition. *Nature Communications*, 16(1), 701. [https://doi.org/10.1038/s41467-025-](https://doi.org/10.1038/s41467-025-56117-0)
266 [56117-0](https://doi.org/10.1038/s41467-025-56117-0)
- 267 Rahnev, D., Balsdon, T., Charles, L., de Gardelle, V., Denison, R., Desender, K., Faivre, N.,
268 Filevich, E., Fleming, S. M., Jehee, J., Lau, H., Lee, A. L. F., Locke, S. M., Mamassian,
269 P., Odegaard, B., Peters, M., Reyes, G., Rouault, M., Sackur, J., ... Zylberberg, A. (2022).
270 Consensus goals in the field of visual metacognition. *Perspectives on Psychological Science*,
271 17(6), 1746–1765. <https://doi.org/10.1177/17456916221075615>
- 272 Rausch, M., Meyen, S., & Hellmann, S. (2025). statConfR: An R package for static models of
273 decision confidence and metacognition. *Journal of Open Source Software*, 10(106), 6966.
274 <https://doi.org/10.21105/joss.06966>
- 275 Steyvers, M., & Peters, M. A. K. (2025). Metacognition and uncertainty communication
276 in humans and large language models. *Current Directions in Psychological Science*.
277 <https://doi.org/10.1177/09637214251391158>
- 278 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
279 Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M.,
280 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ...
281 SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing
282 in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- 283 Yeung, N., & Summerfield, C. (2012). Metacognition in human decision-making: Confidence
284 and error monitoring. *Philosophical Transactions of the Royal Society B: Biological Sciences*,
285 367(1594), 1310–1321. <https://doi.org/10.1098/rstb.2011.0416>